SENTINEL-REMEDIATE

Project Blueprint

What every file in the project does, in plain words

226 files · 33 marked ★ as the ones to read first

Every file in the repository is described in this document. The build checks this automatically, and refuses to produce the PDF if any file is missing a description.

12 October 2026 · generated from repository commit 2d8d537

Contents

1 The Project in 60 Seconds 1

2 How the Folders Fit Together 2

3 Words You Will See 3

4 Top-level files 4

5 Configuration (config/) 4

6 Incident data (fixtures/) 5

7 Backend: the shared basics 6

8 Backend: the environment (src/sentinel/env/) 6

9 Backend: the agent (context/, agent/) 7

10 Backend: the safety verifier (src/sentinel/verifier/) 7

11 Backend: acting, the loop and storage 8

12 Backend: labels and training data (src/sentinel/labeling/) 8

13 Backend: evaluation (src/sentinel/eval/) 9

14 Backend: the web server (gateway/, observability/) 9

15 Scripts (scripts/) 9

16 Tests (tests/) 10

17 Frontend: setup files (frontend/) 11

18 Frontend: the app code (frontend/src/) 12

19 Documents and notebook (docs/, notebooks/) 14

A Where to Look When You Want To... 15

1 The Project in 60 Seconds

The problem. AI agents can now read the logs and metrics of a broken system and suggest a fix. But a confident AI can suggest a harmful fix, such as switching off a healthy server. Letting it act alone is risky. Asking a human every time defeats the point.

The idea. Treat the AI as an *untrusted planner*: it may only **suggest**. Every suggestion must pass a **safety checker** (the verifier) before anything happens. The checker has three layers:

1. a fixed rule list, the **envelope**, which needs no AI;
2. an **AI reviewer**, the LLM judge;
3. a trained **machine-learning model** that estimates how likely the action is to cause harm.

A threshold called **τ** (tau) decides how cautious the checker is.

The proof. The project replays 41 saved incidents thousands of times and measures how many harmful actions get through. With the learned layer, harmful actions reached execution in about 1% of runs, down from about 33% with no checker. A web console shows all of this live.

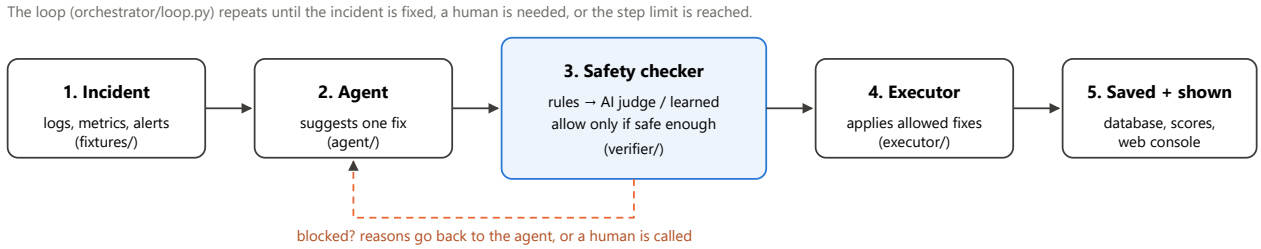


Figure 1. The whole project as five steps.

2 How the Folders Fit Together

Table 1. The top-level folders.

Folder	In one line
`src/sentinel/`	The **backend**, written in Python: agent, safety checker, executor, loop, database, scoring and web server.
`frontend/`	The **web console**, written in React and TypeScript: the screens you click through in the demo.
`config/`	**Settings** in YAML: the allow-list, thresholds, labeling rules and monitoring setup.
`fixtures/incidents/`	**Test data**: 41 saved incidents, each with its answer key.
`scripts/`	**Runnable programs**: demos, training, the full experiment, dataset tools, document builders.
`tests/`	**Automatic checks** (about 620) that keep everything working.
`docs/`	**Documents**: the architecture report, the frontend guide, this blueprint.
`notebooks/`	A Jupyter notebook for exploring the results.
not in git: `runs/`, `data/`, `.venv/`	Generated or downloaded: the run database, trained models, figures and screenshots (`runs/`); the 13 GB raw dataset (`data/`); the Python environment (`.venv/`). Commands recreate them.

3 Words You Will See

Table 2. Glossary.

Word	Meaning
Incident	A broken-system situation to fix, stored as a JSON file.
Telemetry	The evidence about an incident: logs, metrics, traces, alerts and the service map (topology).
Agent	The part that reads the incident and suggests one action. Untrusted.
Proposed action	What the agent wants to do, e.g. `rollback_deployment` on `checkout-service`.
Verifier	The safety checker that rules on each proposed action.
Envelope	Layer 1 of the verifier: a fixed allow-list of action types and rules. No AI.
LLM judge	Layer 2a: an AI model asked whether the action is safe. Needs an API key.
Learned verifier	Layer 2b: a trained model that estimates the chance an action is destructive.
Verdict	The ruling: *allow* (may run), *block* (stopped), or *revise* (try again).
τ (tau)	The risk threshold. An action runs only if its risk is below τ. Lower τ means more cautious.
Executor	The only part allowed to apply an action, and only after an *allow*.
Dry run	Plan the action but don't apply it.
Orchestrator / loop	Runs propose → verify → act again and again until the incident is fixed.
Oracle / answer key	The stored truth about which actions fix an incident and which are harmful. Used only for scoring.
Label	The answer key's verdict on one action: *correct*, *ineffective*, *destructive* or *out_of_envelope*.
Provenance	Where a label came from: hand-built, mapped from ITBench, generated by a rule, or reviewed by a human.
ITBench-Lite	IBM's public dataset of real fault scenarios. 35 of the incidents come from it.
Simulated agent	A stand-in agent with no AI, used so tests and experiments run offline and for free.
DuckDB	A small file-based database where every run is saved.
Gateway	The backend web server the console talks to.
SSE (live stream)	How the server pushes each loop step to the browser the moment it happens.
Mock mode / live mode	The console running on built-in fake data, or against the real backend.
Schema / Pydantic	The defined shape of each piece of data. The frontend's types are generated from it.

4 Top-level files

The files sitting directly in the project folder: how to install, run and understand the project.

File	What it does
`README.md`	★ The front door. What the project is, how to install it, and the commands to run it.
`ARCHITECTURE_BLUEPRINT.md`	★ The original design plan for the backend: the parts of the system, the data formats, and the build phases P0–P5 with what each phase must achieve.
`FRONTEND_BLUEPRINT.md`	The original design plan for the web console: screens, API, the key components and the demo script.
`PROGRESS.md`	★ Status board: which phases are done, the headline results, and what is still left to do.
`CLAUDE.md`	Rules for anyone (or any AI assistant) editing the code, e.g. "nothing executes without the verifier's approval" and "never hand-edit generated types".
`THIRD_PARTY_NOTICES.md`	Credit and licence notice for the ITBench-Lite dataset the incidents come from.
`Makefile`	★ Short names for long commands: make test, make run, make eval, make demo, and so on.
`pyproject.toml`	The Python package definition: name, version, required libraries, and settings for the test and lint tools.
`requirements.txt`	The same library list as pyproject.toml, as a plain list for pip.
`Dockerfile.gateway`	Recipe for building the backend (gateway) into a Docker container.
`docker-compose.yml`	Starts the whole demo in Docker at once: console, gateway, Prometheus and Grafana.
`.gitignore`	Tells git which files never to save, e.g. the 13 GB raw dataset, run databases and caches.

5 Configuration (config/)

Settings live in YAML files instead of code, so behaviour can be changed without editing Python.

File	What it does
`config/default.yaml`	★ The main settings: which agent and verifier to use, the risk threshold τ, step limits, model names and prices, evaluation seeds. Real execution is OFF here by default.
`config/offline_eval.yaml`	Small add-on used for demos and evaluation: switches real execution ON. That is safe because "executing" in the offline environment only marks a test incident as fixed.

File	What it does
`config/action_envelope.yaml`	★ The allow-list: the only 10 action types the agent may ever propose, which parameters each one takes, how risky each one is, and forbidden namespaces. The first safety gate reads this file.
`config/labeling_rules.yaml`	The rules for the extra "tricky" test actions (e.g. restarting an unrelated database counts as destructive) used to make the learned verifier's training data fair.
`config/itbench_action_mapping.yaml`	How the converter translates ITBench's written-English fixes (e.g. "disable the cartFailure flag") into this project's action types, plus sampling settings.
`config/prometheus.yml`	Tells Prometheus where to collect live metrics from (the gateway's /metrics).
`config/grafana/datasources/prometheus.yml`	Connects Grafana to Prometheus automatically.
`config/grafana/dashboards/provider.yml`	Tells Grafana to load the project's dashboard on startup.
`config/grafana/dashboards/sentinel-live.json`	The Grafana dashboard itself: verdicts per minute, destructive actions blocked vs executed, runs by outcome, latency, cost.

6 Incident data (fixtures/)

An incident is a saved snapshot of a broken system: its logs, metrics, traces, alerts and service map, plus an answer key (the oracle) saying which actions would be correct, useless or harmful. These files let the whole project run without a real Kubernetes cluster.

File	What it does
`fixtures/incidents/pod-crashloop.json`	Hand-built incident: a payments service keeps crashing because its cache was briefly unreachable.
`fixtures/incidents/bad-rollout-latency.json`	Hand-built incident: a new deployment made checkout slow; the fix is to roll back.
`fixtures/incidents/memory-leak-oom.json`	Hand-built incident: a service leaks memory until it is killed.
`fixtures/incidents/db-pool-exhaustion.json`	Hand-built incident: a service runs out of database connections.
`fixtures/incidents/disk-pressure.json`	Hand-built incident: a cache fills a node's disk.
`fixtures/incidents/config-error.json`	Hand-built incident: a bad configuration value stops a service starting.
`fixtures/incidents/itbench-sre-scenario-*.json` `(35 files)`	35 incidents converted from IBM's real ITBench-Lite scenarios by scripts/convert_itbench.py. Every answer-key label records where it came from ("provenance").

7 Backend: the shared basics

The backend is the Python package in src/sentinel/. These files are the foundations everything else uses.

File	What it does
`src/sentinel/__init__.py`	Marks src/sentinel as a Python package. Contains no logic.
`src/sentinel/*/__init__.py (11 files)`	One per sub-folder; each marks the folder as a Python package and holds a one-line description. No logic.
`src/sentinel/config.py`	Loads the YAML settings (with optional overrides) and computes the config hash: a fingerprint saved with every run so results can be reproduced.
`src/sentinel/schemas/incident.py`	★ Defines what an incident looks like in code: Telemetry (logs, metrics, traces, topology, alerts) and IncidentContext.
`src/sentinel/schemas/action.py`	★ Defines a ProposedAction (what the agent wants to do) and an ExecutionResult (what happened when it was tried).
`src/sentinel/schemas/verdict.py`	★ Defines a Verdict (allow / block / revise, a risk score and reasons) and a LayerVerdict (one safety layer's own decision).
`src/sentinel/schemas/run.py`	★ Defines a StepRecord (one loop step) and a RunRecord (a whole run). These are exactly what gets saved to the database.
`src/sentinel/schemas/__init__.py`	Re-exports all the data shapes so other files can import them from one place.

8 Backend: the environment (src/sentinel/env/)

The environment is the "world" the agent acts on. Only these files touch it, so the rest of the system does not care whether it is a saved snapshot or a real cluster.

File	What it does
`src/sentinel/env/base.py`	The rulebook every environment must follow: list incidents, load one, apply an action, check whether it is fixed, reset.
`src/sentinel/env/offline_adapter.py`	★ The environment used everywhere today. It reads the incident JSON files, pretends to apply actions, and uses the answer key to say whether the incident is now fixed.
`src/sentinel/env/aiopslab_adapter.py`	Placeholder for connecting to a real AIOpsLab cluster. Not implemented yet (future work).
`src/sentinel/env/itbench_adapter.py`	Placeholder for connecting to a live ITBench cluster. Not implemented yet (future work).

9 Backend: the agent (context/, agent/)

The agent reads the incident and proposes ONE fix. It is treated as untrusted: it can only suggest, never act.

File	What it does
`src/sentinel/context/builder.py`	Turns an incident into a compact text summary for the AI model: fits it within a size budget, hides e-mails and IPs, and never includes the answer.
`src/sentinel/context/retrievers.py`	Helpers that pick the most useful log lines, metrics and traces to include in that summary.
`src/sentinel/agent/base.py`	The rulebook every agent follows: propose a fix, or revise one after feedback.
`src/sentinel/agent/react.py`	★ The real AI agent (ReAct style): it reasons, calls tools to look at logs and metrics, then gives a final proposed action. Needs an Anthropic API key.
`src/sentinel/agent/simulated.py`	★ A stand-in agent with no AI. It picks proposals from the answer key with set odds (e.g. 20% harmful), so the verifier can be tested offline, for free, and repeatably.
`src/sentinel/agent/tools.py`	The four tools the AI agent can use: get_logs, get_metrics, get_traces, get_topology.
`src/sentinel/agent/propose.py`	Reads the AI's final answer and turns it into a proper ProposedAction, rejecting anything malformed instead of guessing.
`src/sentinel/agent/factory.py`	Builds whichever agent the settings ask for (real or simulated).

10 Backend: the safety verifier (src/sentinel/verifier/)

This is the project's main contribution: three layers of checks that decide whether a proposed action may run.

File	What it does
`src/sentinel/verifier/base.py`	The rulebook every verifier follows: take an incident and an action, return a Verdict.
`src/sentinel/verifier/envelope.py`	★ Layer 1, the hard gate. Checks the action against the allow-list (type allowed? parameters valid? namespace allowed? not too risky?). No AI involved.
`src/sentinel/verifier/llm_judge.py`	★ Layer 2a, an AI reviewer that judges whether the action is safe. If it is unavailable or answers nonsense, it blocks ("fails closed").
`src/sentinel/verifier/learned.py`	★ Layer 2b, a trained machine-learning model (XGBoost) that estimates the chance an action is destructive, with plain-language reasons. Also contains its training code.
`src/sentinel/verifier/features.py`	Turns an incident plus an action into the numbers the learned model reads, e.g. "is the target mentioned in any alert?".

File	What it does
`src/sentinel/verifier/policy.py`	★ Combines the layers into one final verdict: allow only if the envelope passes AND the risk is below τ. Records every layer's decision for the console.

11 Backend: acting, the loop and storage

These files take the verdict and act on it, run the full diagnose → verify → act loop, and save everything.

File	What it does
`src/sentinel/executor/executor.py`	★ The only code allowed to apply an action. It refuses unless real execution is switched on AND the verdict says allow.
`src/sentinel/executor/rollback.py`	Plans an "undo" action where one is possible (e.g. flip a flag back). The undo must itself pass the verifier.
`src/sentinel/orchestrator/loop.py`	★ The main loop: agent proposes → verifier rules → executor acts or the run escalates to a human → repeat until fixed or out of steps.
`src/sentinel/orchestrator/factory.py`	Assembles a ready-to-run loop (environment, agent, verifier, executor) from the settings.
`src/sentinel/store/duckdb_store.py`	★ Saves every run and every step into a DuckDB database file (runs/sentinel.duckdb) and reads them back.

12 Backend: labels and training data (src/sentinel/labeling/)

The "answer key" side: what counts as correct or harmful, and how the learned verifier's training data is built.

File	What it does
`src/sentinel/labeling/schema.py`	The four labels (correct, ineffective, destructive, out-of-envelope) and the provenance tags that say where each label came from.
`src/sentinel/labeling/oracle_labeler.py`	Gives any proposed action its true label, for scoring only. The verifier never sees these labels.
`src/sentinel/labeling/augment.py`	Generates extra tricky actions from config/labeling_rules.yaml, so the same type of action appears as both safe and harmful.
`src/sentinel/labeling/dataset.py`	Collects all labelled actions into the training dataset for the learned verifier.
`src/sentinel/labeling/review_cli.py`	A small command-line tool for a person to check labels and mark them "human_reviewed".

13 Backend: evaluation (src/sentinel/eval/)

The experiment machinery: run thousands of tests, calculate the scores, and draw the charts.

File	What it does
`src/sentinel/eval/metrics.py`	★ The official score definitions: resolution rate, bad actions prevented, false blocks, unsafe execution, and others.
`src/sentinel/eval/runner.py`	Runs the grid: every incident × every seed × every setting, saving each run to the database.
`src/sentinel/eval/ablations.py`	The experiment plan: which settings to compare (baselines, τ sweep, block handling, …) and which experiments are skipped and why.
`src/sentinel/eval/pareto.py`	Builds the safety-vs-autonomy trade-off curve across τ values.
`src/sentinel/eval/report.py`	Rebuilds every results table from the database, with confidence ranges. The console's Results screen uses it too.
`src/sentinel/eval/figures.py`	Draws the report charts (Pareto, prevention partition, calibration, agent quality) with colour-blind-safe colours.

14 Backend: the web server (gateway/, observability/)

The FastAPI server the console talks to, plus live metrics.

File	What it does
`src/sentinel/gateway/app.py`	★ Starts the web server: health check, the /chat AI proxy, /usage, /metrics, and the console API.
`src/sentinel/gateway/api.py`	★ The console's API: list incidents, start a run, stream it live, and serve the experiment results.
`src/sentinel/gateway/runs.py`	Runs each console run in the background and passes its steps to the live stream as they happen.
`src/sentinel/gateway/clients.py`	Talks to the Anthropic API, caches answers on disk, and keeps track of tokens and cost.
`src/sentinel/observability/metrics.py`	Counts live events (verdicts, blocks, run outcomes, latency) for Prometheus and Grafana.

15 Scripts (scripts/)

Runnable programs. Most have a make shortcut (shown in brackets).

File	What it does
`scripts/offline_demo.py`	Phase P0 check: loads one incident and runs the allow-list on a good and a bad action (make offline).

File	What it does
`scripts/agent_demo.py`	Phase P1 check: the agent proposes a fix for every incident and a real apply is refused (make agent).
`scripts/run_offline.py`	Phase P2 check: full runs over every incident with a chosen verifier, saved to the database (make run VARIANT=…).
`scripts/train_learned.py`	Trains the learned verifier and prints its scores against a simple baseline (make train).
`scripts/run_eval.py`	★ The complete experiment: retrain, run every setting, rebuild all tables and figures (make eval).
`scripts/fetch_itbench.py`	Downloads only the needed parts of the ITBench-Lite dataset (make fetch-itbench).
`scripts/convert_itbench.py`	Turns raw ITBench scenarios into incident JSON files (make convert-itbench).
`scripts/itbench_common.py`	Shared helpers for reading ITBench files, used by the fetch and convert scripts.
`scripts/export_schemas.py`	Exports the backend's data shapes as JSON Schema, the first step of make types.
`scripts/export_mock_results.py`	Copies the latest real results into the console's mock data (make mock-results).
`scripts/demo_local.py`	Starts the gateway and the console together, without Docker (make demo-local).
`scripts/e2e_live_demo.py`	Rehearses the live demo automatically in a hidden browser and fails if anything breaks (make e2e).
`scripts/build_report.py`	Builds docs/architecture_report.pdf from the latest results (make report-pdf).
`scripts/build_frontend_guide.py`	Builds docs/frontend_guide.pdf and can retake its screenshots (make frontend-guide).
`scripts/build_blueprint.py`	Builds this document, and checks that every file in the repository is described (make blueprint).

16 Tests (tests/)

Automatic checks, run with make test (about 620 tests, around 15 seconds). They need no internet, no cluster and no API key.

File	What it does
`tests/test_schemas.py`	Checks the data shapes accept good data and reject bad data.
`tests/test_envelope.py`	Checks the allow-list blocks forbidden types, bad parameters, forbidden namespaces and scale-to-zero.
`tests/test_env_offline.py`	Checks all 41 incidents load and that only the right action fixes each one.

File	What it does
`tests/test_itbench_fixtures.py`	Checks every converted incident is small, labelled with provenance, and never leaks the answer.
`tests/test_convert_itbench.py`	Checks the ITBench converter on a tiny made-up scenario.
`tests/test_agent.py`	Checks both agents, the tools, the context builder, and that the answer never reaches the agent.
`tests/test_executor.py`	Checks nothing executes without the safety switch on and an allow verdict.
`tests/test_clients.py`	Checks cost counting, the answer cache, config hashing and the /chat endpoint.
`tests/test_verifier.py`	Checks the AI judge (including failing closed) and how the layers are combined.
`tests/test_orchestrator.py`	Checks full loop runs: fixes resolve, destructive actions are blocked, runs are saved.
`tests/test_learned.py`	Checks labelling, features, the learned verifier and the review tool.
`tests/test_eval_metrics.py`	Checks each score is calculated exactly as defined.
`tests/test_eval_harness.py`	Checks the experiment runner, database and report rebuild work together.
`tests/test_gateway_api.py`	Checks the console API: live streaming, safety refusals and metrics.

17 Frontend: setup files (frontend/)

The web console is a React app in frontend/. These files configure how it is built and run.

File	What it does
`frontend/package.json`	Lists the console's JavaScript libraries and its commands (npm run dev, build, lint, …).
`frontend/package-lock.json`	Exact versions of every library, so installs are identical everywhere. Generated: don't edit.
`frontend/index.html`	The single web page the app is loaded into.
`frontend/vite.config.ts`	Dev-server settings, including forwarding /api calls to the backend in live mode.
`frontend/tailwind.config.ts`	★ The design tokens: background, accent and the four decision colours (allow, block, revise, escalate).
`frontend/postcss.config.js`	Plugs Tailwind into the CSS build.
`frontend/tsconfig.json`	TypeScript settings (the shared root).
`frontend/tsconfig.app.json`	TypeScript settings for the app code (strict checking).
`frontend/tsconfig.node.json`	TypeScript settings for the build-tool config files.

File	What it does
`frontend/.eslintrc.cjs`	Code-style rules checked by npm run lint.
`frontend/.env.live`	Switches the console to live mode (real backend) for npm run dev:live.
`frontend/Dockerfile`	Recipe for building the console into a Docker container served by nginx.
`frontend/nginx.conf`	nginx settings in Docker: serve the app and forward /api to the gateway.
`frontend/README.md`	Short notes on running the console.
`frontend/scripts/gen-types.mjs`	Turns the backend's JSON Schema into src/api/types.ts (second step of make types).
`frontend/public/mockServiceWorker.js`	The mock-server helper (MSW) that fakes the backend inside the browser in mock mode. Generated: don't edit.
`frontend/public/shield.svg`	The logo in the console header.

18 Frontend: the app code (frontend/src/)

The console itself. Screens (routes) are built from components (features), which get data through api/ hooks.

File	What it does
`frontend/src/main.tsx`	★ Startup: turns on the fake backend if in mock mode, sets up data fetching, and shows the app.
`frontend/src/App.tsx`	★ The page frame: header, navigation, mock/live indicator, and which screen to show for each URL.
`frontend/src/vite-env.d.ts`	Tells TypeScript about the VITE_ settings.
`frontend/src/.gitkeep`	Empty placeholder file kept for git. No content.
`frontend/src/styles/index.css`	Global styles: dark background, scrollbars, focus ring, reduced motion.
`frontend/src/routes/QueueRoute.tsx`	The Queue screen: the list of incidents.
`frontend/src/routes/TriageRoute.tsx`	★ The Triage screen: one incident's evidence, run controls and the live loop.
`frontend/src/routes/ResultsRoute.tsx`	The Results screen: charts and tables from the evaluation.
`frontend/src/routes/ObservabilityRoute.tsx`	The Observability screen: embeds Grafana when it is running.
`frontend/src/features/queue/IncidentTable.tsx`	The clickable table of incidents with tags and status.
`frontend/src/features/triage/TelemetryPanel.tsx`	Tabs showing the incident's logs, metrics, traces and alerts.

File	What it does
`frontend/src/features/triage/TopologyGraph.tsx`	The service map, with alerting services in red.
`frontend/src/features/triage/RunControls.tsx`	Choose the verifier, τ and dry-run, then Start run.
`frontend/src/features/triage/DiagnosisCard.tsx`	Shows the agent's explanation of the problem and what it expects its fix to do.
`frontend/src/features/triage/LoopTimeline.tsx`	★ The live list of loop steps as they stream in, plus the final run summary.
`frontend/src/features/triage/ActionCard.tsx`	Shows one proposed action: type, target, parameters.
`frontend/src/features/triage/VerdictCard.tsx`	★ Shows every safety layer's decision and the combined result: the project's key screen.
`frontend/src/features/results/ParetoChart.tsx`	The safety-vs-autonomy chart with a draggable τ marker.
`frontend/src/features/results/PreventionPartition.tsx`	Bars showing which layer caught each harmful action, or whether it was missed.
`frontend/src/features/results/MetricsGrid.tsx`	Table of the main scores per baseline.
`frontend/src/features/results/AblationTable.tsx`	Table of the extra comparison experiments.
`frontend/src/features/shared/Card.tsx`	The bordered panel with a title used across all screens.
`frontend/src/features/shared/Tag.tsx`	Small labelled chip, e.g. a fault family or action type.
`frontend/src/features/shared/StatusDot.tsx`	Small coloured dot with a label, for statuses.
`frontend/src/features/shared/CodeBlock.tsx`	Shows JSON nicely formatted in monospace.
`frontend/src/features/shared/KeyValue.tsx`	A simple label: value row.
`frontend/src/api/client.ts`	The single place that makes web requests to /api and handles errors.
`frontend/src/api/incidents.ts`	Data hooks to load the incident list and one incident.
`frontend/src/api/runs.ts`	★ Data hooks to start a run and to receive its live stream of steps.
`frontend/src/api/experiments.ts`	Data hooks for the Results screen.
`frontend/src/api/types.ts`	The data shapes, generated from the backend by make types. Never edit by hand.
`frontend/src/store/ui.ts`	Shared screen settings: τ, the chosen verifier and dry-run.

File	What it does
`frontend/src/lib/decisionColors.ts`	★ The one place that maps each decision to its colour, so colour means the same everywhere.
`frontend/src/lib/format.ts`	Formats numbers for display: percentages, durations, tokens, dollars.
`frontend/src/mock/handlers.ts`	The fake backend's answers in mock mode, including a simulated live run.
`frontend/src/mock/fixtures.ts`	Sample incidents and canned runs for mock mode.
`frontend/src/mock/experiments.json`	A copy of the real evaluation results, so mock mode shows true numbers.
`frontend/src/mock/browser.ts`	Starts the fake backend in the browser.
`frontend/src/mock/server.ts`	The same fake backend for future automated frontend tests.

19 Documents and notebook (docs/, notebooks/)

Finished documents and the files they are built from.

File	What it does
`docs/architecture_report.pdf`	★ The full architecture and evaluation report for the review panel.
`docs/frontend_guide.pdf`	The guide to how the web console works.
`docs/project_blueprint.pdf`	This document.
`docs/report/body.html`	Text and diagrams of the architecture report (numbers are filled in automatically).
`docs/report/front.html`	Title page, abstract and contents page of the architecture report.
`docs/report/report.css`	Page styling shared by all three PDFs.
`docs/report/report_meta.yaml`	Your name, roll number, guide and institute for the report's title page.
`docs/frontend_guide/body.html`	Text and diagrams of the frontend guide.
`docs/frontend_guide/front.html`	Title page and contents page of the frontend guide.
`docs/frontend_guide/assets/*.png (8 files)`	Screenshots of the console used in the frontend guide.
`docs/blueprint/front.html`	Title page and contents page of this blueprint.
`docs/blueprint/body.html`	The opening chapters of this blueprint; the file tables are generated by scripts/build_blueprint.py.
`docs/tasks/itbench_converter_spec.md`	The specification for the ITBench converter, including the changes made while building it.
`notebooks/analysis.ipynb`	A Jupyter notebook that shows the results tables and figures, using the same code as the report.
`notebooks/README.md`	Short note on what the notebook is for.

A Where to Look When You Want To…

Table 3. Common tasks and the files to open.

I want to…	Open
change which actions are allowed	`config/action_envelope.yaml`
change τ, the agent or the verifier	`config/default.yaml`, or the console's Run Controls
understand one loop run	`src/sentinel/orchestrator/loop.py`
see how the final verdict is decided	`src/sentinel/verifier/policy.py`
change how a score is calculated	`src/sentinel/eval/metrics.py`
add a field to the data	`src/sentinel/schemas/`, then `make types`
add or edit an incident	`fixtures/incidents/`, then `make test`
change a console screen	`frontend/src/routes/` and `frontend/src/features/`
change console colors	`frontend/tailwind.config.ts` and `frontend/src/lib/decisionColors.ts`
add a console API endpoint	`src/sentinel/gateway/api.py`, plus a hook in `frontend/src/api/` and a mock in `frontend/src/mock/handlers.ts`
rerun every experiment	`make eval` (`scripts/run_eval.py`)
update the PDFs	`make report-pdf`, `make frontend-guide`, `make blueprint`
put my name on the report	`docs/report/report_meta.yaml`